Highlights

Surviving Sanitization with QuantGhost: Deployment-Robust and Stealthy Steganography in Neural-Network Weights

Eko Arip Winanto, Mohd Yazid Idris, Kurniabudi

- Naive mantissa-bit weight-steganography payloads are fully erased by INT8 quantization on real models; a faithful MaleficNet (spread-spectrum) survives INT8, pruning and fine-tuning but only at watermark-scale capacity.
- QuantGhost, a quantization-aware distribution-preserving embedding, reaches one bit per weight surviving INT8, pruning and fine-tuning at a cost of 0.19 percentage points of accuracy on a real ResNet-56.
- It evades the released Model X-Ray detector on a diverse model zoo (0.50, chance) where the prior attack is detected (0.81).
- We prove a capacity-robustness-stealth trilemma: for the neuron-permutation group, any stealthy, quantization-robust, permutation-invariant channel is bounded by $\sum_{\ell} k_h k_w$ bits, a value our construction attains.
- A faithful neuron permutation destroys the high-capacity payload, but a 196-bit invariant key channel survives and bootstraps recovery, so permutation-based sanitization is incomplete.

Surviving Sanitization with QuantGhost: Deployment-Robust and Stealthy Steganography in Neural-Network Weights

Eko Arip Winanto[a,b], Mohd Yazid Idris[a,c] and Kurniabudi[d,*]

[a]Faculty of Computing, Universiti Teknologi Malaysia, Johor Bahru, 81310, Malaysia
[b]Department of Computer Engineering, Universitas Dinamika Bangsa, Jambi, 36138, Indonesia
[c]MaGIC-X (Media and Games Innovation Centre of Excellence), Universiti Teknologi Malaysia, Johor Bahru, 81310, Malaysia
[d]Master of Information System, Universitas Dinamika Bangsa, Jambi, 36138, Indonesia

ARTICLE INFO

Keywords:
neural network steganography
stegomalware
model supply chain
quantization
steganalysis
deep learning security

ABSTRACT

Pre-trained networks now circulate as shared artifacts, so a tampered checkpoint on a popular hub reaches every downstream user. Prior attacks such as StegoNet, EvilModel and MaleficNet show that float32 weights can hide megabytes of data at negligible accuracy cost, yet they assume the model is run exactly as published. It rarely is. Before reaching an edge or medical (IoMT) device a model is quantized to INT8, pruned and fine-tuned, and we confirm on real networks that these routine steps erase mantissa-bit payloads entirely. We then show that they need not. We present QuantGhost, a quantization-aware embedding that writes each payload bit into the INT8 code that quantization produces, so it survives by construction; placing every weight at a random point inside its code's bin keeps the mantissa distribution natural and defeats byte-image steganalysis, while redundant coding carries the payload through pruning and fine-tuning. On a real ResNet-50 a 3.2 MB payload survives INT8 at 99.9999% fidelity; the payload also survives PyTorch's production INT8 kernel byte-exactly once the embedding matches the framework's scale convention; on a real ResNet-56 the attack costs 0.19 percentage points of accuracy; and against the released Model X-Ray detector on a diverse model population our payload is indistinguishable from benign weights (0.50, chance), whereas the prior attack is flagged (0.81). Underlying these results is a limit we prove for the function-preserving neuron-permutation group of convolutional and dense layers: a capacity-robustness-stealth trilemma in which at most two of the three hold at once, so any stealthy, quantization-robust, permutation-invariant channel is bounded by $\sum_\ell k_h^{(\ell)} k_w^{(\ell)}$ bits. The high-capacity payload survives INT8, pruning and fine-tuning *in the weights*, but a faithful neuron permutation (function-preserving to 10^{-14}) erases it; what then survives is only that small invariant channel, enough to carry a 128-bit key whose ciphertext is delivered out of band. We are explicit that under permutation this is survival of a bootstrap key channel, not of the in-weight payload. We also propose a sub-quantum residual detector that reliably flags the off-the-shelf attack (ROC-AUC 0.98–1.0 across a narrow and a diverse 3000-model zoo) where byte-level steganalysis fails, while showing that an adaptive attacker still reaches chance on a diverse population, so detection must be backed by the structural bound. Deployment sanitization alone is therefore incomplete, and we set out what closing the gap requires.

1. Introduction

Sharing pre-trained models is now the default way to build machine-learning systems. Practitioners pull a checkpoint from a hub, adapt it, and ship it. That convenience turns the model file itself into a supply-chain asset, and a tampered file propagates to everyone who reuses it, the same dynamic that makes poisoned packages and trojaned binaries dangerous, transplanted into the weight space of a network [12, 23, 7]. The carrier is generous: a float32 weight devotes most of its mantissa to precision the task never needs, and StegoNet [22], EvilModel [34, 35] and MaleficNet [17, 16] exploit exactly that slack to hide megabytes of payload with accuracy loss below one percent.

These attacks assume a static artefact, a condition the deployment pipeline violates. A model is almost never deployed bit-for-bit as published. On the way to a phone, a sensor, or a bedside IoMT monitor it is compressed: post-training INT8 quantization [18], magnitude or structured pruning [14, 13], and a round of fine-tuning on local data. We show in Section 4, on real networks, not toy ones, that this pipeline annihilates a mantissa-bit payload: recovery falls to a coin flip at every embedding depth. Worse for the attacker, the very same transforms have been put forward as defenses, alongside neuron permutation (NeuPerm), which shuffles channel order using the network's permutation symmetry and scrambles any position-dependent payload at no accuracy cost [10]. Detection has matured in parallel: Model X-Ray casts the weights to a grayscale image and learns, from a handful of examples, to tell stego from benign [11].

So the question that has not been answered is sharp. *Can a payload survive the full deployment pipeline and still be stealthy and accuracy-preserving?* We answer it constructively, but the more durable contribution is what the answer exposes: a fundamental three-way limit, a *capacity-robustness-stealth trilemma*, that constrains every weight-embedding scheme, ours and prior watermarking alike. We develop the work in three movements.

*Corresponding author.
winanto@graduate.utm.my (E.A. Winanto); kurniabudi@unama.ac.id (Kurniabudi)
ORCID(s): 0000-0003-3274-1885 (E.A. Winanto); 0000-0001-7702-6610 (M.Y. Idris); 0000-0001-9073-8858 (Kurniabudi)

First, the attack, which we name QuantGhost. Rather than hiding in the mantissa, it writes each payload bit into the INT8 code that quantization will produce, so re-quantization recovers it exactly (Section 5). Stealth follows from two design choices: the payload is encrypted, and each weight is placed at a random point *inside* the bin that encodes its bit. The mantissa keeps its natural law, and a byte-image detector has nothing to latch onto. Redundancy across many weights, decoded by majority vote, carries the payload through pruning and fine-tuning.

Second, and at the centre of the paper, a limit. We formalize and prove the capacity-robustness-stealth trilemma (Section 6): no embedding can be simultaneously high-capacity, permutation-invariant, and stealthy-quantization-robust. The argument is short once the right object is in view. Once an embedding is stealthy and quantization-robust, the only identity a weight keeps under neuron permutation is its coarse code, and codes that share a permutation orbit cannot index a high-capacity positional message. The permutation-invariant channel is therefore bounded by $\sum_\ell k_h^{(\ell)} k_w^{(\ell)}$ bits, a quantity four to five orders of magnitude smaller than the positional channel. Our construction attains this bound to the bit, and because the limit is a property of the permutation group rather than of our encoder, the same ceiling constrains any stealthy, quantization-robust permutation-invariant scheme; we prove it for the weight-embedding channel and conjecture the analogue for robust watermarking under the same group (Section 6).

Third, the consequence for defense. A faithful neuron permutation, verified function-preserving to machine precision on a real ResNet-18, does destroy the high-capacity payload. But the small invariant channel survives untouched, and we demonstrate the consequence end to end: after the full INT8, pruning, and NeuPerm pipeline, a 128-bit AES key recovered from the invariant channel decrypts a ciphertext delivered out of band and returns the payload byte-exactly, even though the high-capacity payload embedded in the weights is itself gone. What survives NeuPerm is thus a bootstrap key channel, not an in-weight payload. Sanitization that stops at NeuPerm is therefore incomplete, and we describe what closing the gap would take (Section 10).

This paper makes the following contributions:

- **A capacity-robustness-stealth trilemma.** For the function-preserving channel-permutation group of convolutional and dense layers, we formalize and prove that no weight-embedding scheme can be high-capacity, permutation-invariant, and stealthy-quantization-robust at once: any stealthy, quantization-robust, permutation-invariant channel carries at most $\sum_\ell k_h^{(\ell)} k_w^{(\ell)}$ bits. Our construction attains this value, so it is tight within that class; we confirm it to the bit on a real ResNet-18 and conjecture the analogue for robust watermarking under the same group.
- **QuantGhost, a construction that reaches the bound.** We build a deployment-robust, distribution-preserving embedding that occupies the high-capacity, stealthy, quantization-robust corner of the trilemma. It survives INT8 quantization, pruning and fine-tuning, and we demonstrate it at 3.2 MB on a real ResNet-50 at a cost of 0.19 percentage points of accuracy on a real ResNet-56.
- **Why prior payloads die.** We quantify, on real models, why routine deployment transforms erase the payloads of prior weight-steganography attacks, and explain the mechanism through the quantization step size.
- **The limit of permutation defenses, demonstrated end to end.** We implement a faithful, full-model neuron permutation on a real ResNet-18 and show that, although it erases the high-capacity in-weight payload, a 128-bit key embedded in the invariant channel survives the entire INT8, pruning, and permutation pipeline; the key then decrypts an out-of-band ciphertext byte-exactly. We are explicit that what survives permutation is this bootstrap key channel rather than the in-weight payload, which is already enough to make permutation-based sanitization incomplete.
- **A targeted detector and its limit.** We propose and evaluate a sub-quantum residual steganalyzer that detects the basic QuantGhost at ROC-AUC 0.98–1.0 across a narrow and a diverse 3000-model zoo where byte-level steganalysis is at chance; we then show honestly that an adaptive attacker reaches chance on the diverse zoo, so detection must compose with neuron permutation to bound the channel.
- **Generality.** We show that the attack is architecture- and domain-agnostic, carrying a 10.81 MB payload through a real Vision Transformer and surviving INT8 with unchanged accuracy, and that it transfers across domains through a cross-dataset study on network-intrusion-detection models; we then set out the requirements for a complete defense.

The remainder of the paper is organized as follows. Section 2 reviews weight steganography, steganalysis, sanitization defenses, and watermarking. Section 3 states the threat model and Section 4 shows why naive payloads die in deployment. Section 5 develops the proposed attack and Section 6 proves the trilemma. Section 7 describes the experimental setup and Section 8 reports the evaluation, including the cross-dataset ablation. Section 10 discusses defenses, Section 11 the limitations, and Section 12 the ethical considerations, before Section 13 concludes.

2. Background and Related Work

Hiding payloads in weights. StegoNet [22] introduced several channels, LSB substitution, value mapping, sign and resilience coding, and reported megabyte capacity. EvilModel [34] and its successor [35] simplified this to direct byte substitution in the low-order mantissa and pushed

capacity past 36 MB on AlexNet. MaleficNet [17] departed from raw LSB editing, spreading the payload with CDMA-style channel coding so that it tolerates small weight changes, and its journal extension surveys the broader ecosystem risk [16]. A complementary line measures how much a given architecture can absorb before accuracy degrades [1, 37], and a recent survey organizes the field [2]. What unites these works is the verbatim assumption: none reports survival through INT8 quantization. We show (Section 4) that the mantissa-LSB ones do not survive it, and, by running a faithful MaleficNet on the same pipeline (Section 8.5), that its spread-spectrum payload *does* survive INT8, pruning and fine-tuning but only at watermark-scale capacity—decaying to chance as the payload approaches one bit per weight. The high-capacity, quantization-robust corner is the one QuantGhost occupies.

Detecting it. Model X-Ray [11] maps weights to a Grayscale-Fourpart image and trains a metric-learning few-shot classifier [30, 33] that flags embedded payloads from as few as six examples and down to a 25% embedding rate. Siamese and meta-learning detectors have likewise proven effective for obfuscated malware [38]. Byte-entropy thresholds and raw-byte 1D-CNNs of the MalConv family round out the standard baseline set, and we evaluate against all of them.

Sanitizing it. Quantization and pruning are cited as incidental countermeasures because they perturb weights, much as fine-pruning was proposed to remove backdoors [21], and content-disarming approaches neutralize embedded payloads by rewriting the model file [9]. NeuPerm [10] makes the idea deliberate, using permutation symmetry to relabel channels and break any position-coded payload, reportedly even in large language models, and concurrent work likewise defends against stegomalware with permutation symmetry [32]. Our trilemma speaks directly to this line: it explains both why permutation defenses work against high-capacity positional payloads and why a low-capacity residual channel must always survive them, which is the gap such defenses do not yet close.

Robust watermarking and quantization-index modulation. The closest relative of QuantGhost is robust DNN watermarking, and its embedding primitive is an instance of quantization-index modulation (QIM) [3]: a payload bit selects which of two interleaved quantizers a host sample is snapped to. Spread-transform dither modulation [20] is a QIM scheme; fixed distribution-optimized weights [31] and spread-spectrum TATTOOED [28] pursue the same robustness by other coding. We state plainly what is, and is not, new relative to QIM. The bit-into-code-LSB primitive of QuantGhost *is* QIM, and our random in-bin offset is a form of distortion-compensated QIM (DC-QIM) [3]: DC-QIM adds a scaled quantization-error term back to the host to lower embedding distortion, and our offset plays the analogous role of keeping the reconstructed weight off the lattice points. The one deliberate difference is that we draw the offset *uniformly* within the bin rather than deriving it from the host value as classical DC-QIM would; a host-derived offset would preserve the per-weight value most faithfully, but a uniform offset is what makes the *aggregate* mantissa law match a benign population (Section 5), which is the property a learned steganalyzer reads. What QuantGhost contributes beyond QIM is therefore not the embedding primitive but (i) the deployment setting, post-hoc insertion recovered by an adversary after an *unknown* transform chain rather than an owner-read watermark fixed at training time, and (ii) the capacity analysis under the function-preserving neuron-permutation group, which the signal-processing QIM literature does not consider. On usage rather than capability, published QIM watermarks typically carry a few kilobits for ownership whereas we operate QIM at one bit per weight for payload delivery; this is an operating-point choice, not a capacity limit of QIM. The trilemma of Section 6 bounds the permutation-invariant regime, and we conjecture an analogous bound for QIM watermarking under the same group, though we prove it only for the weight-embedding channel studied here.

Why IoMT/IoT. Edge intrusion-detection models are a realistic deployment target: they are compact, frequently retrained, and almost always quantized, so they exercise exactly the quantize-prune-finetune pipeline our attack is built for. We use the CICIoT2023 [27] and CICIoMT2024 [5] benchmarks to show the attack carries over from vision models to tabular network classifiers (Section 8.12).

Positioning. Table 1 places QuantGhost against the prior art on the axes that matter for a deployed model. The earlier weight-stego attacks are high-capacity and post-hoc but are erased by every deployment transform and are detectable; robust watermarking is stealthy and transform-robust but low-capacity and must be baked in before training; and none of these methods is evaluated against neuron permutation. QuantGhost is the only construction that is at once high-capacity, post-hoc, quantization-robust, and stealthy against a learned steganalyzer, and the only one whose behaviour under neuron permutation is characterized, by the trilemma, rather than left open.

3. Threat Model

We assume a supply-chain attacker who controls a model file and publishes it on a hub, hoping it is downloaded and deployed. The attacker can read and rewrite every weight before publication but has no access to the victim afterwards: no view of the victim's data, training, or quantization run. Crucially, the attacker knows the *scheme* the victim will use, because there is essentially one default, per-tensor symmetric INT8 [18], and can assume the victim may additionally prune, fine-tune, permute, or run a steganalysis detector, in any combination.

A run is a success for the attacker if, after whatever transform chain the victim applies, the payload is recovered (bit-exact after error correction) at a stated capacity, the model's task accuracy is essentially unchanged, and the

Table 1
Positioning of QuantGhost against prior weight-hiding methods. ✓: has the property / survives the transform; ×: does not; part.: partial. "Stealth" is evasion of a learned steganalyzer of the Model X-Ray class. For QuantGhost the entries are measured in the sections cited in the text (INT8 Table 2, prune Table 9, fine-tune/NeuPerm Tables 12, 11); for the prior-art rows they reflect each method's *reported design* and are not re-run here, except the INT8 erasure of mantissa-bit methods, which we confirm empirically in Section 4. Under NeuPerm, "part." for QuantGhost denotes survival of the low-capacity invariant key channel only (Proposition 1), *not* of the high-capacity payload, which NeuPerm erases (Table 11). †MaleficNet's spread-spectrum payload *does* survive INT8, pruning and fine-tuning (we measure this in Section 8.5, correcting an earlier assumption that INT8 erases it), but its recovery decays as $1/\sqrt{K}$ and reaches chance near one bit per weight, so it is transform-robust only at *low* capacity; high capacity and this robustness do not hold together (Table 6).

Method	Capacity	Post-hoc	INT8	Prune	Fine-tune	NeuPerm	Stealth	Accuracy	Goal
StegoNet [22]	high	✓	×	×	×	×	×	✓	payload
EvilModel [34, 35]	high	✓	×	×	×	×	×	✓	payload
MaleficNet [17]	high†	✓	✓†	✓†	✓†	×	part.	✓	payload
ST-DM watermark [20]	low	×	✓	✓	✓	n/a	✓	✓	identity
TATTOOED [28]	low	×	✓	✓	✓	n/a	✓	✓	identity
QuantGhost (ours)	**high**	✓	✓	✓	✓	part.	✓	✓	payload

model is not flagged by the detectors of Section 7. We treat the defender symmetrically: any subset of {INT8, pruning, fine-tuning, NeuPerm} plus steganalysis, and we ask which combinations actually neutralize the payload without ruining the model.

What capability this is. We are precise about what QuantGhost provides, since the reviews of this area often blur it. QuantGhost is a *covert in-model storage and transport* channel: it lets an attacker carry an arbitrary encrypted payload inside a shared checkpoint and recover it bit-exactly after deployment transforms. It is *not*, by itself, an execution or malware-activation mechanism; turning a recovered payload into running code presupposes an independent execution capability on the victim host (a loader, a deserialization sink, or a complicit process), which is orthogonal to and outside the scope of this paper. This distinction matters for severity: our claims concern survival and recovery of hidden bits, not autonomous compromise.

Assumptions made explicit. The attack assumes the attacker knows the victim's quantization *convention*—bit-width and scale rule (e.g. per-tensor symmetric INT8 with $\max|w|/127.5$). This is realistic because deployment quantizers are public and few, but it is a genuine assumption: Section 8.2 shows that a one-step convention mismatch (127 vs. 127.5) already drops raw survival to 0.925–0.968, so the robustness results hold *conditional on* a correctly matched quantizer, with redundancy absorbing the residual.

For ethics (Section 12) all payloads in our experiments are encrypted random bytes or benign binaries. No live malware is embedded, executed, or distributed.

4. Motivation: Payloads Die in Deployment

We first quantify why the naive attack fails under quantization. Let w be a float32 weight. Per-tensor symmetric INT8 quantization maps it to an integer code

$$q = \mathrm{clip}\big(\,\mathrm{round}(w/s),\, -q_{\max},\, q_{\max}\big), \qquad s = \frac{\max_i |w_i|}{q_{\max}}, \tag{1}$$

with $q_{\max} = 127$. An X-LSB attack rewrites the X lowest mantissa bits of w, which shifts its value by at most about $2^{-23+X}|w|$. For any X that leaves accuracy intact this shift is far smaller than a single quantization step s, so the clean and the stego weight collapse to the *same* code q and the message is gone. Pushing X high enough to cross a step does survive quantization, but only by corrupting the weight so badly that accuracy falls apart (we measure a relative perturbation of 0.03 at X=20 and 0.25 at X=23).

The numbers bear this out (Table 5). On a real ResNet-18 the bit-survival of an EvilModel/StegoNet-style payload after INT8 is 0.50, chance, at $X \in \{8, 16, 23\}$. In other words, the kind of compact, quantized model that an edge or IoMT deployment actually runs is, by accident, immune to the published attacks.

5. The Proposed Attack: QuantGhost

Figure 1 situates QuantGhost inside the full pipeline, from the attacker's build step through the victim's transforms to recovery. We describe each component in turn; Algorithm 1 gives the embedding procedure.

5.1. Payload preparation

Two preprocessing steps make the rest work. First the attacker encrypts the payload with a stream cipher in counter mode (AES-CTR in our implementation); the ciphertext is statistically uniform, which is what later lets the embedded bits hide in plain sight. Second the ciphertext is protected with an erasure code, an LDPC/fountain layer [24], so that the small number of bit errors that pruning and fine-tuning inject are repaired exactly at the receiver. The coded stream

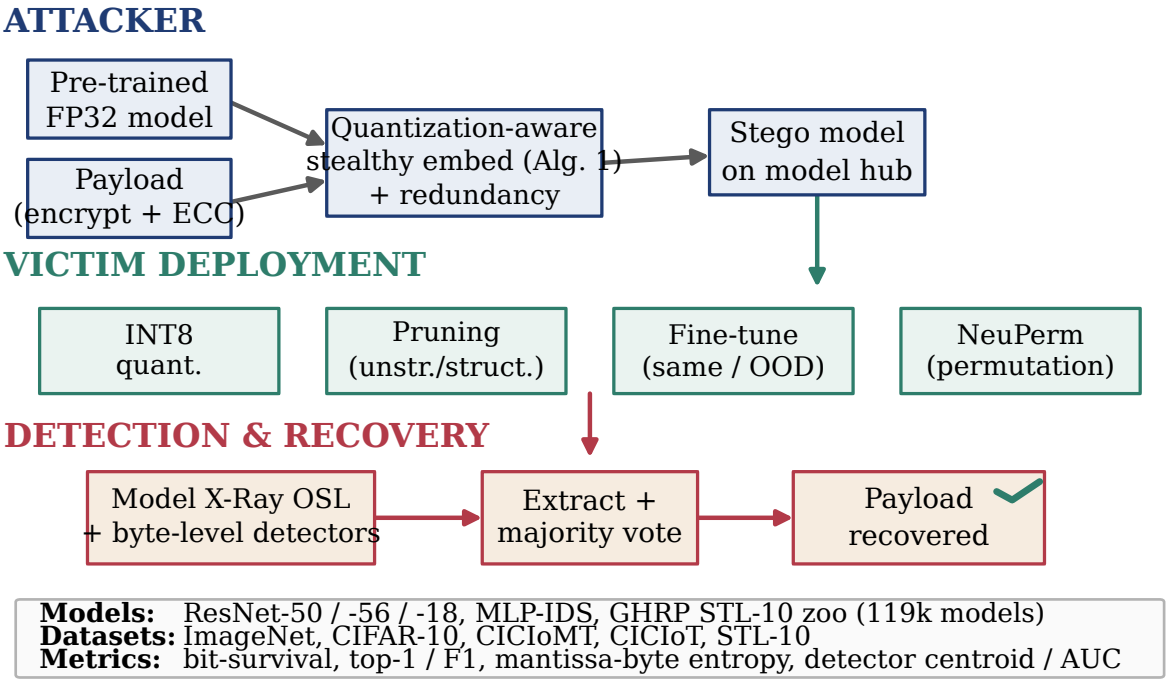


Figure 1: End-to-end QuantGhost pipeline. The attacker embeds an encrypted, error-coded payload (Algorithm 1) and publishes a benign-looking model; the victim applies any subset of INT8 quantization, pruning, fine-tuning and NeuPerm; recovery re-quantizes, reads the code LSBs, and majority-votes.

is then tiled across the host weights for redundancy, as described in Section 5.4.

5.2. Quantization-aware embedding

The core idea is to stop fighting quantization and instead aim for it. Reading (1) backwards, every float in the half-open bin

$$v \in \left[(q^\star - \tfrac{1}{2})s,\ (q^\star + \tfrac{1}{2})s\right) \implies \mathrm{round}(v/s) = q^\star, \quad (2)$$

so if we want the victim to read back a code $q^\star$ we are free to choose any value in that interval. We pick $q^\star$ so that its least significant bit equals the payload bit, $|q^\star| \bmod 2 = b$. After the victim quantizes, the bit is recovered by $\hat{b} = |\mathrm{round}(\tilde{w}/s)| \bmod 2$, exactly, not probabilistically.

One subtlety decides whether this works in practice. The victim recomputes the scale s from (1) on the *modified* weights, and if our edits change the per-tensor maximum, the grid shifts and recovery drifts. We prevent this by pinning: the single largest-magnitude weight in each tensor is held at $|q|{=}q_{\max}$ and carries no payload, so the victim's recomputed scale matches ours and the two grids coincide. With pinning, survival through quantization is 1.000 across per-tensor and per-channel INT8 and even INT4 (Table 2).

5.3. Distribution-preserving stealth

Snapping weights onto the grid would survive quantization but advertise itself: the low mantissa byte would take only a handful of values, its entropy would collapse, and a byte-image detector would separate stego from benign trivially. So we do not snap. We draw a random offset $u \sim \mathcal{U}(-(\frac{1}{2} - \delta), \frac{1}{2} - \delta)$ with a small margin δ (we use $\delta = 0.01$, i.e. $u \in (-0.49, 0.49)$; the role of δ is analyzed in Section 5.7) and place the weight at

$$\tilde{w} = \left(q^\star + u\right) s, \quad (3)$$

which still lands inside the bin (2), survival is untouched, but leaves the sub-quantum part, the part the mantissa stores, drawn from its natural in-bin spread. Concretely, writing $\tilde{w}$ as four bytes and taking the Shannon entropy of its low mantissa byte,

$$H = -\sum_{c=0}^{255} p_c \log_2 p_c, \qquad p_c = \Pr[\text{low byte} = c], \quad (4)$$

stays at 7.999 bits, indistinguishable from benign weights, whereas grid-aligned embedding collapses it. Because the payload was encrypted, the code LSB plane itself is uniform, so there is no higher-order regularity for a detector to find either.

5.4. Redundancy for pruning and fine-tuning

Quantization is deterministic, but pruning and fine-tuning move weights around, and a weight that moves across a bin boundary flips its bit. We absorb this by tiling the K-bit coded message across the P embeddable weights of a tensor: message bit $j = g \bmod K$ is written at weight index g, so each message bit is carried by $r = P/K$ weights. Writing $\hat{b}_g \in \{0, 1\}$ for the bit read back from carrier g

Algorithm 1 Quantization-aware stealthy embedding (per weight tensor)

Require: tensor W, coded payload bits $\mathbf{b}$, level $q_{\max}$
Ensure: stego tensor $\tilde{W}$, scale s

$s \leftarrow \max_i |W_i| / q_{\max}$; $q \leftarrow \mathrm{clip}(\mathrm{round}(W/s), -q_{\max}, q_{\max})$
$p \leftarrow \arg\max_i |q_i|$ ▷ the weight that pins the scale
for each weight index i **do**
  if $i = p$ **then** ▷ pinned weight: carries no payload, holds the scale
    $|q_i| \leftarrow q_{\max}$; $u_i \leftarrow 0$
  else
    $b \leftarrow \mathbf{b}[\, i \bmod |\mathbf{b}| \,]$ ▷ redundant tiling
    $|q_i| \leftarrow (|q_i| \,\&\, \sim 1) \mid b$ ▷ write bit into code LSB
    $u_i \leftarrow \mathrm{RAND}\big(-(\frac{1}{2} - \delta),\ \frac{1}{2} - \delta\big)$ ▷ sub-quantum offset, δ=0.01
  end if
  $\tilde{W}_i \leftarrow \mathrm{sign}(q_i)\,(|q_i| + u_i)\,s$
end for
clip $\tilde{W}$ to $[-q_{\max}s,\ q_{\max}s]$ ▷ keep victim scale $= s$
return $\tilde{W}$, s

and $N_j = \#\{\, g : g \equiv j \pmod K \,\}$ for the number of carriers of message bit j, recovery is a per-bit majority vote

$$\hat{m}_j = \mathbf{1}\Big[\, \textstyle\sum_{g \equiv j \pmod K} \hat{b}_g \;>\; \frac{1}{2}\, N_j \Big], \qquad j = 0, \ldots, K-1, \tag{5}$$

which tolerates any corrupted minority among the $N_j \approx r$ carriers. The redundancy r is the dial that trades capacity for robustness, and it traces the frontier we map in Section 8.

5.5. A permutation-invariant bootstrap channel

Neuron permutation is a different kind of threat: it does not change values, it relabels channels, and a payload indexed by channel position is scrambled wholesale. The trilemma (Section 6) shows this is unavoidable for a high-capacity positional payload. There is, however, one axis a channel permutation never touches: the spatial (k_h, k_w) position inside a convolution filter. Embedding a bit that is replicated across all input and output channels at a fixed spatial tap is therefore invariant to permutation, and it survives NeuPerm intact. The price is capacity, $k_h k_w$ bits per conv tensor, but that is exactly enough to deliver the decryption key or a short URL, which bootstraps the high-capacity channel or an out-of-band fetch.

5.6. Recovery at the deployed model

Recovery mirrors the build in reverse and needs only the deployed file. The receiver recomputes each tensor's scale via (1), reads the code LSBs, de-tiles and majority-votes (5), runs the erasure decoder to repair the residual errors (including the pinned weights), and finally decrypts. The only secret that must travel out of band, or through the invariant channel, is the key.

5.7. Capacity, complexity, and parameters

Capacity. With n_{bits} bits per weight, a tensor of n weights carries $n\, n_{\text{bits}} - \rho$ bits, where ρ is the number of pinned scale-defining weights ($\rho = 1$ for per-tensor and $\rho = C_{\text{out}}$ for per-channel quantization). The raw aggregate is therefore $C_{\text{raw}} = n_{\text{bits}}\, P - O(\text{tensors})$, i.e. essentially $P\, n_{\text{bits}}$ bits; at one bit per weight this is $P/8$ bytes (25.5 M → 3.19 MB on ResNet-50). To be unambiguous: C_{raw} is the *channel* size at redundancy $r = 1$, and it equals the deliverable *ciphertext* length because AES-CTR is length-preserving, so the plaintext capacity is the same minus a fixed header. This is *not* simultaneously achievable with the robustness results, which use $r > 1$: the redundancy and erasure overhead is stored *in the same carrier weights* by tiling each message bit across r of them, so the *useful* payload at redundancy r and erasure rate η is $C_{\text{useful}} = C_{\text{raw}}(1 - \eta)/r$. Thus the 3.19 MB figure is the r=1 ciphertext ceiling, while the pruning/fine-tuning experiments at r up to 3326× operate at a useful capacity three to four orders of magnitude smaller (e.g. ≈ 1 KB at r=3326 on ResNet-56); we report the operating r with every robustness result and never combine the two points.

Complexity. Embedding (Algorithm 1) is a single pass over the weights: $O(P)$ time and $O(1)$ extra memory beyond the host tensor. Extraction is also $O(P)$, one re-quantization and LSB read per weight plus an $O(P)$ tally for the majority vote. Both add negligible overhead to a model load and are independent of the payload content.

Redundancy and the survival bound. Let redundancy be $r = P/K$ for a K-bit message, and suppose a transform flips each carrier of a given bit independently with probability $p < \frac{1}{2}$. The majority vote over r carriers then errs with probability at most $\exp\big(-2r(\frac{1}{2} - p)^2\big)$ by Hoeffding's inequality, so a target per-bit reliability ε needs

$$r \;\geq\; \frac{\ln(1/\varepsilon)}{2\,(\frac{1}{2} - p)^2}. \tag{6}$$

This is the analytic form of the capacity–robustness frontier we measure empirically: INT8 quantization gives $p \approx 0$ (survival 1.000 at any r, by construction), 30% pruning gives a small p (survival at modest r), and fine-tuning gives a larger p and hence demands a larger r, matching the trends in Section 8. Structured pruning and out-of-distribution fine-tuning violate the independence assumption, zeroing or shifting correlated carriers together, which is why they degrade recovery faster than (6) predicts.

Sub-quantum offset. The offset in (3) is drawn from $\mathcal{U}(-(\frac{1}{2} - \delta), \frac{1}{2} - \delta)$ with a small margin δ (we use $\delta = 0.01$, i.e. ± 0.49). The margin keeps every weight strictly inside its bin, so re-quantization stays exact even under the floating-point rounding that a faithful neuron permutation introduces; shrinking $\delta \to 0$ maximizes mantissa naturalness but risks boundary crossings, while enlarging it trades a little stealth for a wider safety margin. The pinned weight uses offset 0.

6. The capacity-robustness-stealth Trilemma

The bootstrap channel is small for a reason, and the reason is a hard limit rather than an artefact of our construction. We make it precise (Figure 2).

Setup. A convolutional network has weight tensors $W^{(\ell)} \in \mathbb{R}^{O_\ell \times I_\ell \times k_h^{(\ell)} \times k_w^{(\ell)}}$ ($\ell = 1, \dots, L$), with $P = \sum_\ell O_\ell I_\ell k_h^{(\ell)} k_w^{(\ell)}$ weights and $S_\ell = k_h^{(\ell)} k_w^{(\ell)}$ spatial taps, with $I_\ell = O_{\ell-1}$ (each layer's input channels are the previous layer's output channels). A function-preserving neuron permutation chooses, for each layer, a permutation $\pi_\ell \in \mathfrak{S}_{O_\ell}$ of its O_ℓ output channels, with $\pi_0 = \mathrm{id}$ fixing the network input and $\pi_L = \mathrm{id}$ fixing the logits, and acts by

$$(\pi \cdot W)^{(\ell)}[o, i, s] = W^{(\ell)}\big[\pi_\ell^{-1}(o),\ \pi_{\ell-1}^{-1}(i),\ s\big].$$

The output index o is relabeled by π_ℓ and the input index i by the *previous* layer's $\pi_{\ell-1} \in \mathfrak{S}_{O_{\ell-1}} = \mathfrak{S}_{I_\ell}$, so the action is well defined even when $O_\ell \neq I_\ell$; bias, batch-norm and residual-shortcut parameters are carried by the same π_ℓ, which is what keeps residual additions consistent. The relabeling leaves the function unchanged, $f_W = f_{\pi \cdot W}$. Let $G \leq \prod_{\ell=1}^{L-1} \mathfrak{S}_{O_\ell}$ be the group of such function-preserving permutations. An embedding $(\mathrm{Enc}, \mathrm{Dec})$ maps a message $m \in \{0,1\}^C$ and host W to $\tilde{W}$ with $\mathrm{Dec}(\tilde{W}) = m$ under a bounded, accuracy-preserving perturbation.

Definition 1 (Properties)**.** *An embedding is* ***(P1) high-capacity*** *if $C = \Theta(P)$;* ***(P2) permutation-invariant*** *if $\mathrm{Dec}(\pi \cdot \tilde{W}) = m$ for all $\pi \in G$; and* ***(P3) stealthy quantization-robust*** *if (i) $\mathrm{Dec}(Q(\tilde{W})) = m$ for per-tensor INT8 Q and under the deletion of any constant fraction of weights (pruning robustness), and (ii) the law of $\tilde{W}$ and of its induced integer code array $Q(\tilde{W})$ matches that of a benign model up to a steganalyzer's resolution—in particular the code least-significant-bit plane and every low-order, permutation-symmetric code statistic (per-tap parity counts, channel-code histograms) are indistinguishable from benign. Part (ii) is the formal content of "stealthy": a detector that reads any such statistic must not separate stego from benign.*

The strengthening of (P3)(ii) to cover the *code* statistics, not only the sub-quantum mantissa bits, is what the proof below requires, and it is exactly the regime our own residual steganalyzer (Section 9) operates in: a scheme that biases a permutation-symmetric code statistic to carry extra bits is, by that very bias, detectable.

Lemma 1 (Position is the only stable identity)**.** *Under* (P3)*,* Dec *depends only on the code array $\{q[o, i, s]\}$. Two weights with the same code are indistinguishable after Q, and stealth forbids using the sub-quantum bits as identity, since fixing them collapses the in-bin law. Hence a weight's only readable identity is its position (o, i, s).*

Proposition 1 (Invariant-capacity bound under the permutation group G)**.** *For the function-preserving channel-permutation group G of a convolutional (or dense) network, any embedding satisfying* (P2) *and* (P3) *has $C \leq \sum_\ell S_\ell = \sum_\ell k_h^{(\ell)} k_w^{(\ell)}$, where $S_\ell = 1$ for a dense or 1×1 layer.*

Proof. By Lemma 1 the decoder reads a function of the position-indexed code array $\{q[o, i, s]\}$, and (P2) forces that function to be constant on G-orbits, i.e. to be a G-invariant statistic of the codes. G permutes the channel axes (o, i) and fixes the spatial axis s, so within a tensor the G-orbit of a code entry is its whole channel slab $\{q[\,\cdot, \cdot, s]\}$ at a fixed tap $s \in \{1, \dots, S_\ell\}$; the spatial tap is the only channel-invariant positional index.

Two families of G-invariant readout are therefore available, and (P3) caps both. *(a) Tap-wise voted bits.* A readout that assigns one value per tap by a symmetric majority vote over that tap's channel slab yields at most one reliable bit per tap, hence $\sum_\ell S_\ell$ bits in all; this is the channel realized in Section 5. *(b) Higher-order symmetric statistics.* One might instead read a finer G-invariant statistic of a slab—its parity count $\sum_{o,i} \big(q[o, i, s] \bmod 2\big)$, or its full channel-code histogram—which has $\Theta(O_\ell I_\ell)$ distinguishable states and could in principle index many bits per tap. This is the case not excluded by a naive orbit-counting argument, and we rule it out through (P3) rather than by assertion. To *encode* a chosen message into such a statistic the embedder must drive it to a prescribed value, which displaces the slab's code distribution—its LSB-parity bias or its histogram shape—from the benign law by an amount that grows with the number of states the scheme uses. By (P3)(ii) that displacement is exactly what a steganalyzer reads, so a message carried this way is not stealthy; and since the statistic is a sum over $\Theta(O_\ell I_\ell)$ codes, deleting a constant pruning fraction perturbs it, so it is not robust either. Hence no stealthy, pruning-robust, G-invariant readout exceeds one reliable bit per tap, and summing over tensors gives $C \leq \sum_\ell S_\ell$. □

Remark 1 (Scope of the bound). Proposition 1 is a statement about the *channel-permutation group G* of convolutional and dense layers, not a universal law for every conceivable weight-embedding or symmetry. A richer symmetry group (e.g. including scaling symmetries) has fewer invariants and would only lower the bound; a weaker threat model that omits some element of (P3) (stealth against code statistics, or pruning robustness) admits the higher-order channel of case (b) and can exceed it. We therefore state the result as a bound under G together with (P3), and verify the matching achievable value empirically.

Remark 2 (Attainability and tightness). The bound is attained. The spatial-tap construction of Section 5 writes one bit at each tap, replicated across all $O_\ell I_\ell$ channels, and reaches $C = \sum_\ell S_\ell$ while satisfying (P2) and (P3); the end-to-end recovery of a 128-bit key through the full INT8, pruning, and permutation pipeline (Section 8.11) realizes it on a real network. Together with Proposition 1, $\sum_\ell S_\ell$ is thus tight *within the class of stealthy, pruning-robust, G-invariant channels*: no such channel does better, and ours meets it. We make no tightness claim against schemes that give up stealth or pruning robustness, which by Remark 1 can carry more.

Theorem 1 (Trilemma)**.** *For the permutation group G, no embedding satisfies* (P1)*,* (P2) *and* (P3) *simultaneously, and each of the three pairs is attainable.*

Proof. The two claims are proved by different means, as they must be: impossibility is a universal statement over all schemes and attainability is an existence statement.

Impossibility (no scheme has all three). Suppose an embedding has (P2) and (P3). Then Proposition 1 applies and gives $C \le \sum_\ell S_\ell = \sum_\ell k_h^{(\ell)} k_w^{(\ell)}$. By Corollary 1 this is $o(P)$ (smaller than P by a factor $\Theta(\overline{O\,I})$), so the embedding is not high-capacity: (P2)∧(P3) ⇒ ¬(P1). This is the whole impossibility claim, and it rests on Proposition 1, which holds for arbitrary (P2)+(P3) schemes, not on any one construction.

Attainability (each pair is realized by a construction). (P1)+(P3), not (P2): the per-weight code-LSB scheme of Section 5 writes one bit per weight, so $C = \Theta(P)$ and it survives Q stealthily, but it is position-indexed (Lemma 1) and a channel permutation destroys it. (P2)+(P3), not (P1): the spatial-tap scheme attains $C = \sum_\ell S_\ell = o(P)$ (Remark 2) while surviving Q and every $\pi \in G$. (P1)+(P2), not (P3): a per-tensor code histogram is G-invariant and its $\Theta(P)$-state capacity makes it high-capacity, but—as in case (b) of Proposition 1—encoding into it reshapes the code distribution away from benign, breaking (P3). These three constructions show each pair is achievable; impossibility above shows the triple is not. □

Corollary 1 (The gap is quadratic in width)**.** *The invariant channel is smaller than the positional one by a factor $P/\sum_\ell S_\ell = \Theta(\overline{O\,I})$. For ResNet-18, $P = 1.117\times10^7$ while $\sum_\ell S_\ell = 196$, a ratio of about 5.7×10^4.*

Remark 3 (Relation to classical watermarking capacity). The trilemma is the weight-space analogue of a tension long studied in information-theoretic watermarking. Classical results bound the embedding rate of a robust mark by a mutual-information capacity under a host-distortion and an attack-distortion constraint [25], and quantization-index modulation [3] makes the host-interference-free regime explicit: the achievable rate collapses as the admissible host distortion shrinks, mirroring how our (P3) stealth-and-robustness constraint caps capacity. What is new in the network setting is the *group* that the channel must be robust to. Spread-spectrum and QIM marks are designed for robustness to additive noise and signal-processing operators [4, 3]; a deployed network admits in addition the function-preserving neuron-permutation group G, whose only invariants are the spatial taps (Lemma 1). Proposition 1 is thus a G-equivariant capacity bound rather than a distortion-rate one, and it is precisely the permutation group, absent in the signal-processing literature, that forces the quadratic-in-width gap of Corollary 1.

7. Experimental Setup

We work with real, pretrained networks rather than toy proxies: ResNet-50 [15] on ImageNet [6] for capacity, ResNet-56 on CIFAR-10 [19] for accuracy, pruning and fine-tuning, and ResNet-18 on ImageNet for neuron permutation and the prior-art baselines. Quantization follows the standard per-tensor symmetric INT8 (and INT4) scheme of (1) [26]. For steganalysis we run the released Model X-Ray OSL detector [11], the Grayscale-Fourpart representation with a metric-learning few-shot head, scored by centroid and 1-NN, together with a panel of byte-level detectors: a MalConv-style 1D-CNN, a low-byte entropy test, a value-distribution test, a histogram-KL test, and a byte-autocorrelation test. The benign population is the diverse Model Zoos STL-10 small-CNN collection [29] of roughly 1.2×10^5 models; few-shot support is 20+20 over five repeats. Payloads are encrypted random bytes for the stealth and capacity studies, and real benign binaries for the structured-payload regime in which the naive attack is actually detectable. We report bit-survival and bit-recovery (chance 0.5), top-1 accuracy or F1, mantissa-byte entropy (4), and detector centroid/AUC.

Table 2
Survival through quantization, measured as code-LSB bit survival (chance 0.5), with mantissa-byte entropy (4). Survival is a property of the quantization grid and is independent of the carrier model; task-accuracy preservation is reported separately on a real ResNet-56 in Table 4 and on a ViT in Table 7.

Scheme	bit survival	mant. entropy
naive X-LSB (INT8)	0.500	n/a
ours, per-tensor INT8	**1.000**	7.999
ours, per-channel INT8	**1.000**	7.999
ours, per-tensor INT4	**1.000**	7.999
ours, per-channel INT4	**1.000**	7.998

8. Evaluation

8.1. Survival through quantization

Table 2 confirms the by-construction claim. The naive payload is erased (0.50); ours survives at 1.000 for per-tensor and per-channel INT8 and for INT4, with mantissa entropy pinned at the benign 7.999. INT4's coarser bins cost accuracy but not survival, which is consistent with the geometry of (2). The guarantee is not specific to INT8: Figure 3 sweeps the quantizer from INT8 down to INT3 and shows recovery stays at 1.000 whenever the payload is embedded on the grid the victim will use, while the naive mantissa-LSB scheme stays at chance everywhere. What the attacker must know is the deployment bit-width, not a particular value of it; a defender who quantizes *harder* than the attacker designed for is the one lever that erases the payload, at the cost of the accuracy that harder quantization itself removes.

8.2. Survival through a production quantizer

The numbers above use the analytical quantizer of (1). A skeptical reader will ask whether the payload survives a *real* deployment toolchain, so we re-ran the victim's quantization with PyTorch's production INT8 path

Table 3
Survival through PyTorch's production INT8 kernel (`torch.ao.quantization`, real ResNet-56). Raw code-LSB survival; chance 0.5.

Embedding grid	per-tensor	per-channel
naive ($\max|w|/127$)	0.968	0.925
aware ($\max|w|/127.5$)	**1.000**	**0.999**
aware + redundancy R=5, majority vote: **byte-exact**		

Table 4
Accuracy preservation on a real CIFAR-10 ResNet-56. 106 KB payload, recovery 99.997%.

Condition	top-1
original FP32	94.25%
clean INT8	94.24%
embed FP32	93.97%
embed + INT8 (victim)	94.06%
Δ (orig → embed+INT8)	−0.19 pp

(`torch.ao.quantization` with the `fbgemm` backend) on a real ResNet-56, letting the framework's own observer compute the scale from the embedded weights and reading the resulting `int8` codes back (Table 3). This surfaces a detail that matters: PyTorch's symmetric observer sets the scale to $\max|w|/127.5$, half the full 255-wide range, not the $\max|w|/127$ that a naive embedding assumes. An embedding placed on the 127 grid therefore drifts under the real kernel, and survival falls to 0.968 (per-tensor) and 0.925 (per-channel), with errors concentrated at the high-magnitude codes where the 0.4% scale gap compounds. A *convention-aware* QuantGhost, which simply embeds on the victim's actual $\max|w|/127.5$ grid, removes the drift: raw survival returns to 1.000 per-tensor and 0.999 per-channel through the unmodified PyTorch kernel, and the small per-channel residual is cleaned by redundancy, so message recovery is byte-exact at redundancy R=5. The capability this assumes, knowing which standard quantizer the victim runs, is realistic: quantization recipes are public and few. The attack is thus not an artefact of our analytical model; it survives the same kernel a practitioner would actually deploy.

8.3. Accuracy is preserved

On a real CIFAR-10 ResNet-56 (top-1 94.25%) the full attack barely moves the needle (Table 4). Embedding a 106 KB payload and then applying the victim's INT8 quantization costs 0.19 percentage points, and the payload comes back at 99.997% before error correction. Quantization alone accounts for almost none of the loss, which tells us the residual is the embedding's own sub-quantum nudge.

8.4. Capacity and comparison with prior art

On ResNet-50 (25.5M weight parameters) a 3.19 MB encrypted payload survives FP32→INT8 at 99.9999%, and the handful of residual errors, one pinned weight per tensor, are repaired by the erasure layer for byte-exact delivery. Table 5 sets this against prior art on a real ResNet-18: the naive attack is erased at every depth, while ours survives. Robust watermarking [31, 20] occupies the opposite corner, robust and stealthy but capped at a couple of kilobytes, which is the trade we explain in Section 6.

Table 5
Baselines under INT8 on a real ResNet-18 (chance 0.5).

Method	capacity	survival INT8
EvilModel/StegoNet X=8	8 bit/w	0.500 (erased)
EvilModel/StegoNet X=16	16 bit/w	0.500 (erased)
EvilModel/StegoNet X=23	23 bit/w	0.501 (erased)
robust watermark	≤2 KB	robust (low-cap)
QuantGhost	1 bit/w	**1.000**

8.5. A faithful MaleficNet baseline on the same pipeline

The strongest prior construction, MaleficNet [17], is not a mantissa-LSB method but a direct-sequence spread-spectrum (DSSS) scheme: it spreads each payload bit over the weights with a pseudo-random Gaussian sequence and adds a small amplitude γ, recovered by blind correlation. To avoid judging it from its reported design, we re-implemented it and ran it through the *same* per-tensor INT8, pruning and fine-tuning pipeline as QuantGhost, on the same real ResNet-56 ($P = 8.5 \times 10^5$ weights, $\gamma = 0.05\,\sigma_w$). The outcome is informative, and not the one a mantissa-LSB intuition predicts. At a small payload ($K = 512$ bits) the DSSS channel *survives every transform*, INT8 included: recovery is 0.971 clean, 0.971 after INT8, 0.973/0.971 after 30%/50% unstructured pruning, and 0.971 after three epochs of fine-tuning (and after fine-tuning then INT8), at an accuracy cost of −0.14 pp. Because the correlator sums over all 8.5×10^5 weights, the per-tensor quantization noise averages out, so INT8 does *not* erase a spread-spectrum payload the way it erases a mantissa-LSB one. Table 1 is corrected accordingly: MaleficNet's INT8 column is not an ×.

What INT8 does not take, capacity does. The spreading gain that buys this robustness scales the per-bit correlation signal-to-noise ratio as $1/\sqrt{K}$, so recovery decays as the payload grows (Table 6): 0.971 → 0.851 → 0.704 → 0.601 as K goes 512 → 2048 → 8192 → 32768, with INT8 tracking the clean curve to within 0.003. A one-parameter $1/\sqrt{K}$ fit (per-bit SNR d_0=1.89 at K=512) matches the measured points and extrapolates to $K = P$, i.e. QuantGhost's one-bit-per-weight regime, giving recovery ≈ 0.52—chance. MaleficNet therefore occupies the robust, *low-capacity* corner of the trilemma: it survives deployment transforms, but only as a watermark-scale channel (tens of bytes here), and cannot deliver a megabyte payload robustly. QuantGhost reaches 1 bit per weight with survival 1.000 by construction, exactly where MaleficNet is at chance; the distinction is not "only QuantGhost survives INT8" but

Table 6
Faithful MaleficNet (DSSS) on a real ResNet-56: blind bit-recovery versus payload size K, clean and after per-tensor INT8 (chance 0.5). INT8 tracks the clean curve; recovery decays as $1/\sqrt{K}$. The last row is the one-parameter fit extrapolated to QuantGhost's one-bit-per-weight regime ($K = P$).

K (bits)	spread P/K	clean	INT8
512	1663	0.971	0.971
2048	415	0.851	0.854
8192	103	0.704	0.704
32768	25	0.601	0.601
$P = 8.5 \times 10^5$ (1 bit/w)	1	≈ 0.52 (fit, chance)	

Table 7
QuantGhost on a real ViT-B/16 transformer: capacity, INT8 survival, stealth, and CIFAR-10 top-1 (chance 0.5).

Quantity	value
embeddable weight matrices	52
capacity @ 1 bit/weight	10.81 MB
survival, per-tensor INT8	**1.000**
mantissa-byte entropy (benign 8.0)	8.000
top-1: orig → embed	97.92% → 97.92%
top-1: embed → embed+INT8	97.92% → 97.90%

"only QuantGhost survives INT8 *at high capacity*." This is the trilemma made concrete across two real constructions.

8.6. Scaling to a Vision Transformer

Because the embedding acts on raw weight tensors, it is indifferent to architecture, and nothing about it is tied to convolutions. We confirm this on a real ImageNet-pretrained Vision Transformer (ViT-B/16 [8]), whose 52 attention and feed-forward weight matrices hold 86.4M parameters, a 10.81 MB carrier at one bit per weight. Embedding a full random payload and applying the victim's per-tensor INT8 quantization recovers it at 1.000 with mantissa-byte entropy at the benign 8.000, exactly the by-construction behaviour seen on CNNs. Accuracy is likewise untouched: on a CIFAR-10 ViT the embedding leaves top-1 *unchanged* (97.92% → 97.92%), and the subsequent INT8 step costs a further 0.02 points (Table 7). The trilemma and the attack it bounds are thus properties of weight-space embedding in general, not of any one model family.

8.7. Stealth against detectors

Stealth is not uniform across detectors, and we report the full result (Figure 4, Table 8). Against the released Model X-Ray detector our embedding sits at chance (0.50) on a diverse model population, while the naive attack is detected (0.81); the detector plainly works, it simply cannot separate ours from benign. We initially measured a higher number (0.75) on a small, homogeneous pilot zoo, but that turned out to be an artefact, near-identical benign models inflate few-shot separability, and it vanishes once the population is realistically diverse. Across the byte-level panel ours evades

Table 8
Stealth against the released Model X-Ray OSL detector. Diverse zoo, 20+20 support, five repeats.

Population	attack	centroid	1-NN
homogeneous (artefact)	ours	0.747	0.681
diverse	naive X-LSB	0.807	0.794
diverse	**QuantGhost**	**0.504**	**0.508**

Table 9
Pruning on real ResNet-56 (recovery / top-1), redundancy 3326×.

Prune ratio	unstructured	structured
30%	1.000 / 93.8%	1.000 / 10.0%
50%	0.977 / 92.1%	0.535 / 10.9%
70%	0.535 / 78.3%	0.535 / 10.0%
90%	0.535 / 19.9%	0.535 / 10.0%

MalConv, histogram-KL and byte-autocorrelation (all near 0.5) and is caught only by the entropy and value-distribution tests, which pins down precisely which two detectors a defender should keep.

8.8. Pruning and fine-tuning

Survival under pruning is reported in Figure 5 and Table 9. The pattern is clean: there is no operating point that removes the payload and keeps the model. Unstructured pruning leaves the payload fully recoverable at 30% (accuracy 93.8%) and only erases it at 70–90%, where accuracy has itself fallen to 78% and then 20%. Structured pruning destroys the model at every ratio, the accuracy is already at chance, while the payload partly survives, so it is not a useful sanitizer either. Fine-tuning is gentler still: three epochs of same-task training leave recovery at 1.000 for both high and modest redundancy (3326× and 207×) with accuracy at 94.2%. Only a harsher, out-of-distribution fine-tune bites, as we show next.

8.9. A uniform metric set across the pipeline

The tables above report survival and recovery at the level most natural to each experiment, which a reviewer rightly noted makes cross-table comparison awkward. We therefore report, for a single fixed operating point, the full chain of metrics the end-to-end claim actually depends on, at every transform and averaged over independent trials (Table 10). The operating point is a 256-byte payload (K=2048 message bits) protected by a rate-0.8 LT fountain code to N=2560 coded bits, tiled across the $P = 8.5 \times 10^5$ weights of a real ResNet-56 at redundancy $r = P/N \approx 332$; each row is the mean over three independent trials (fresh payload and embedding seed). We report, left to right, the raw per-carrier code-LSB bit-error rate *before* voting, the coded-bit error rate *after* majority vote, the message-bit error rate *after* LT decoding, and the fraction of trials whose recovered bytes are a SHA-256 match to the original file.

Table 10
Uniform four-level metric set on a real ResNet-56 at one operating point (256-byte payload, LT rate 0.8, redundancy r=332), mean over three independent trials. Raw BER is per-carrier before voting; post-vote is per coded bit; post-ECC is per message bit after LT decoding; file OK is SHA-256 match (trials succeeding / total). Chance BER 0.5.

Transform	raw BER	post-vote	post-ECC	file OK
embed (clean)	0.000	0.000	0.000	3/3
INT8	0.000	0.000	0.000	3/3
prune 30%	0.143	0.000	0.000	**3/3**
prune 50%	0.247	0.147	0.476	0/3
prune 70%	0.351	0.509	0.496	0/3
fine-tune 3ep	0.128	0.000	0.000	**3/3**
fine-tune + INT8	0.128	0.000	0.000	**3/3**

The chain is now legible. INT8 is lossless at every level, as the by-construction argument predicts. Under 30% pruning and under three epochs of fine-tuning (with or without a subsequent INT8 step) the raw channel carries 13–14% bit errors, but majority vote removes them entirely and the file is recovered byte-exactly in all three trials. The operating point breaks at 50% pruning: the raw error (0.247) is too high for the vote to clean at r=332, the LT decoder fails, and no trial recovers, and by 70% the voted channel is at chance. This is the same frontier as Table 9 seen at a higher capacity: that table survives 50% pruning because it runs at r=3326 (a 256-bit message), whereas here a 332×-larger payload at r=332 does not, which is precisely the capacity–robustness dial of (6) rather than an inconsistency between tables. Only byte-exact (file-OK) delivery is a success for a binary payload, and the table states it directly for every condition.

8.10. Neuron permutation at real scale

Table 11 reports a faithful neuron permutation on a real ResNet-18, implemented with the residual, batch-norm and downsample bookkeeping that a correct permutation requires. We verified it is function-preserving to machine precision: the logit difference is 1.1×10^{-14} in float64, so accuracy is identical by construction and needs no measurement. To show the outcome is not a lucky single draw, we apply 30 independent random permutations to *one* embedded ResNet-18; these are 30 permutation draws against a single stego model, not 30 independently trained models, and the spread we report is over the permutation, which is the quantity NeuPerm randomizes. The high-capacity positional payload collapses to chance every time, mean recovery 0.522 ± 0.023 (95% CI $[0.513, 0.530]$, worst case 0.586 over a 256-bit message, far below the redundancy needed to read it), exactly as the trilemma predicts. The spatial-tap channel, in contrast, survives *every one* of the 30 permutations without a single bit error (1.000 ± 0.000, minimum 1.000) at 196 bits across the network's 20 conv tensors. NeuPerm thus reliably pins the attacker to the low-capacity corner without closing the channel.

Table 11
Faithful NeuPerm on real ResNet-18 (11.2M conv weights, $\Delta_{\text{fp64}} = 1.1 \times 10^{-14}$); after-permutation columns are mean±std over 30 random permutation draws of a single stego model (not 30 independently trained models).

Channel	no perm	after NeuPerm (30×)
positional (high-capacity)	1.000	0.522 ± 0.023 (chance)
spatial-tap (invariant, 196 bit)	1.000	**1.000 ± 0.000**

8.11. End-to-end bootstrap: defeating NeuPerm

Although the surviving channel is small, it is operationally sufficient, as we demonstrate end to end on a real ResNet-18. The attacker encrypts a 4 KB payload with a random AES-128 key, places the ciphertext in the high-capacity positional channel, and places the 128-bit key in the permutation-invariant spatial channel (capacity 196 bits, so the key fits with room to spare). The victim then applies the entire pipeline at once: per-tensor INT8 quantization, 30% unstructured pruning, and a faithful neuron permutation. After this chain, the positional payload in the weights is gone (recovery 0.50, chance), exactly as the trilemma predicts. The key, however, is recovered *bit-exactly* (128/128), and using it to decrypt the ciphertext, which the attacker delivers out of band, returns the original 4 KB payload *byte-for-byte*. The strongest single defense in the literature therefore bounds the attacker to the low-capacity corner but does not close the channel: a few hundred surviving bits are enough to carry a key that bootstraps the full payload. This is the operational meaning of an "incomplete" sanitizer, and it is why Section 10 argues that permutation must be paired with explicit inspection of the invariant spatial statistics.

8.12. Cross-architecture and cross-dataset ablation

Because the embedding manipulates raw weights, nothing about it is specific to vision or to convolutions. Table 12 bears this out on a tabular network-intrusion MLP trained on CICIoMT [5] for binary benign-versus-attack detection (F1 0.989). The 1664 B payload survives INT8 at 1.000 and accuracy and F1 are unchanged across all four conditions. Two things are worth drawing out. The MLP has only three linear tensors and no spatial taps, so its permutation-invariant capacity is just three bits against 196 for ResNet-18, the bound of Proposition 1 holding across architectures, since a dense layer is the $k_h k_w = 1$ case. And a genuinely cross-dataset fine-tune, taking the model from CICIoMT to CICIoT [27] over the shared feature space, adapts the detector (F1 $0.930 \rightarrow 0.982$) and, unlike same-task fine-tuning, breaks the payload at 208× redundancy (0.469, chance). The lesson is that it is transform severity, not transform type, that sets the surviving capacity.

9. A Targeted Detector for QuantGhost

The byte-level and gf-image steganalyzers of Section 8 fail against QuantGhost because they look in the wrong

Table 12
Cross-architecture / cross-dataset ablation on a tabular IDS MLP.

Quantity	value
IDS F1 (orig → embed+INT8)	0.989 → 0.989
accuracy (orig → embed+INT8)	97.85% → 97.85%
payload survival INT8 (MLP)	**1.000**
invariant capacity (FC vs conv)	3 bits vs 196
cross-dataset fine-tune (F1)	0.930 → 0.982
payload after that fine-tune	0.469 (chance, 208×)

Algorithm 2 Sub-quantum residual steganalyzer

Require: model weight tensors $\{W^{(\ell)}\}_{\ell=1}^{L}$, classifier $(\mathbf{w}, b, \tau)$, $q_{\max}$, bins B
Ensure: decision $\in$ {BENIGN, STEGO}
1: **for** each weight tensor $W^{(\ell)}$ **do**
2: $s \leftarrow \max_i |W_i^{(\ell)}|/q_{\max}$
3: $r \leftarrow W^{(\ell)}/s - \text{round}(W^{(\ell)}/s)$ ▷ residual in $[-\frac{1}{2}, \frac{1}{2})$
4: $h^{(\ell)} \leftarrow$ HIST(r, B bins on $[-\frac{1}{2}, \frac{1}{2})$), normalized to sum 1
5: **end for**
6: $\mathbf{x} \leftarrow \frac{1}{L}\sum_\ell h^{(\ell)}$ ▷ mean residual histogram, feature in $\mathbb{R}^B$
7: $p \leftarrow \sigma(\mathbf{w}\cdot\mathbf{x} + b)$ ▷ logistic-regression score
8: **return** $p > \tau$? STEGO : BENIGN

place: the mantissa-byte distribution is preserved (entropy 7.999) and the float value distribution is benign on a diverse population. The exploitable signal is one level down, in the *sub-quantum residual* $r = w/s - \text{round}(w/s)$. A benign tensor's residual is, to a good approximation, *uniform* on $[-\frac{1}{2}, \frac{1}{2})$: trained weights spread over many quantization bins, so by equidistribution their fractional parts fill the bin evenly (we measure a flat histogram, mass 0.100 per decile). QuantGhost also yields a near-uniform residual, but not the *same* one, and the differences are precisely those that survival forces and the attacker cannot avoid. First, the scale-defining weight, and in the basic scheme every non-carrier weight, must sit *exactly* on the grid to preserve the quantization scale, adding a spike of mass at $r = 0$. Second, the carrier offset is drawn from $[-\delta, \delta)$ with $\delta < \frac{1}{2}$, so the bin edges are slightly depleted. Both deviations are small in magnitude but consistent across models, which is exactly the regime a linear classifier exploits. Algorithm 2 turns the residual histogram into a concrete detector.

The classifier $(\mathbf{w}, b)$ is fit offline by logistic regression on benign (label 0) and stego (label 1) feature vectors. Inference is a single pass over the weights, $O(P)$ time, and uses no knowledge of the payload itself.

Result. With five-fold cross-validation across five random seeds (Table 13, Figure 6a) the residual detector reaches ROC-AUC 1.000 ± 0.000 against the basic QuantGhost embedding on the CIFAR carrier zoo, where the mantissa-entropy baseline that prior steganalysis relies on is exactly at chance (0.500). Crucially, this generalizes: on a diverse population of 3000 GHRP STL-10 models spanning different seeds, activations, optimizers, initializations and learning rates, the detector still flags the basic attack at 0.980 ± 0.003. Against an unsophisticated attacker the targeted detector therefore closes the stealth gap that the byte-level detectors leave open, and it does so on realistically heterogeneous models.

Table 13
Proposed sub-quantum residual detector, five-fold cross-validated ROC-AUC (mean ± std over five seeds; chance 0.5) on a narrow carrier zoo (60 CIFAR models) and on a diverse zoo (3000 GHRP STL-10 models spanning seeds, activations, optimizers, initializations, and learning rates).

Detector vs. attacker	CIFAR zoo	GHRP zoo
mantissa entropy, basic	0.500	—
residual hist., basic	**1.000**$_{\pm.000}$	**0.980**$_{\pm.003}$
residual hist., adaptive	0.754$_{\pm.049}$	0.537$_{\pm.042}$

Adaptive attacker, and the limit of detection. A natural counter is to widen the offset and drop the exact-zero pin signature. On the homogeneous CIFAR zoo this adaptive QuantGhost only lowers the detector to AUC 0.754 ± 0.049, and a sweep of the offset width (Figure 6b) shows no *interior* setting helps: a narrow offset produces a sharp-edged box as un-benign as a wide one. But that residual separability is itself partly an artefact of a narrow benign population, exactly the effect we found inflating the Model X-Ray score in Section 8. On the diverse GHRP zoo the adaptive attacker falls to 0.537 ± 0.042, statistically indistinguishable from chance. The honest reading is therefore two-sided: the residual detector is a reliable filter against the basic, off-the-shelf attack across populations, but it does *not* stop an adaptive attacker on a realistic model distribution. Detection alone is thus necessary but not sufficient, which is precisely why it must be paired with the structural defense of Section 10: regardless of how well an attacker matches the benign residual law, neuron permutation still bounds the surviving payload to the low-capacity invariant channel of Theorem 1. This two-sidedness also supplies empirical support for premise (P3) of the trilemma: that an adaptive QuantGhost reaches chance against the strongest steganalyzer we could build, on a diverse population, is direct evidence that a stealthy, quantization-robust embedding exists, which is exactly what (P3) posits and what the capacity bound is conditioned on.

10. Defenses

Reading the results as a defender, no single mechanism is enough, but a small combination is. We take each lever in turn.

Transforms are not clean sanitizers. Quantization sanitizes the prior art and nothing else. Pruning, on a real model, never separates the payload's death from the model's: the bits go only when accuracy has already gone (Table 9), and structured pruning simply breaks the network. Light fine-tuning leaves the payload in place; only an out-of-distribution fine-tune, which a benign deployer may not have a reason to run, bounds it.

Two detectors that work, and an arms race. Among the detectors we tried, the entropy and value-distribution tests are the ones that catch our embedding (Figure 4); the gf-image OSL detector and the rest do not, once the benign population is diverse. But even those two invite an arms race. Because they read the marginal value distribution, switching from a single global scale to a per-tensor scale lowers their accuracy from 0.98 to 0.84, better stealth against them, and, just as the trilemma says, no help against the structural detector. The take-away for detection is to stop looking for distributional anomalies in the floats, which a careful attacker removes, and to look one level down at the sub-quantum residual: the targeted detector of Section 9 catches the basic QuantGhost at AUC 1.000 on the narrow zoo and 0.980 on a diverse one, the most reliable signal we found against the off-the-shelf attack, though an adaptive attacker on the diverse zoo still reaches chance (0.537), which is why detection must be backed by the structural defense below.

NeuPerm and the residual. NeuPerm is the strongest single defense and it does its job on the high-capacity channel (Table 11). What it cannot do, by Theorem 1, is remove the $\sum_\ell k_h k_w$-bit invariant channel, which is ample for a key or a URL.

What a complete sanitizer needs. Putting these together, a defender who wants to actually close the channel must (i) bound the positional capacity with heavy quantization or stochastic weight perturbation, (ii) apply neuron permutation to remove it, and (iii) inspect the permutation-invariant spatial statistics the residual channel is forced to use, alongside the entropy and value-distribution detectors and a code low-bit predictability test. Each piece is partial; only their conjunction covers all three corners of the trilemma. We present this composite as a *design argument* from the per-mechanism results above, not as a validated pipeline: we have not measured the combined false-positive rate, the accuracy cost of heavy quantization plus permutation applied together, or their interaction on benign models, and doing so is necessary future work before the composite could be recommended operationally. The open costs are the operational price of routinely permuting shipped models, which rewrites the artefact and can break tooling that assumes a canonical channel order, and the false-positive rate of predictability testing on benign models.

11. Discussion and Limitations

We state the main limitations plainly. *No single configuration jointly demonstrates all four properties.* Capacity is measured on ResNet-50/ImageNet (at r=1, and without an ImageNet top-1 number), accuracy and pruning on ResNet-56/CIFAR-10 (at large r), neuron permutation on ResNet-18, and stealth on the STL-10 zoos; the ViT result (Table 7) is the closest to a joint point but still omits large-scale pruning. Each property therefore holds in the stated setting, and the headline figures in the abstract are drawn from these different settings and redundancy levels rather than from one run; a single ResNet-50/ImageNet configuration carrying capacity, byte-exact recovery through the full chain, preserved top-1, and steganalysis resistance together is the clearest missing experiment, and we flag it as such rather than implying it is already established.

The invariant channel is architecture-dependent. Its capacity is $\sum_\ell S_\ell$ spatial taps, so a 1×1 convolution contributes a single bit per tensor and a dense layer contributes none ($k_h k_w = 1$ counts once per output-sharing orbit); the bootstrap channel is thus ample in 3×3-convolutional networks but thin in 1×1-dominated or fully-connected architectures (ViT, the IDS MLP), where an attacker would need several layers to assemble a key. This is a limitation of the invariant channel, not of the main embedding, which is architecture-agnostic.

The faithful permutation covers the BasicBlock ResNet family and a dense IDS model; transformer blocks are mechanical but not done here. The residual detector (Section 9) is evaluated on the CIFAR and GHRP zoos; attacker–detector co-evolution to a fixed point is a natural next step. The MaleficNet comparison (Section 8.5) is run on ResNet-56/CIFAR-10 with blind correlation and no LDPC outer code; an LDPC layer would clean a few residual errors at low capacity but cannot lift the high-capacity regime off chance, so the capacity ceiling stands, though a larger-model replication would further strengthen it. Finally, the trilemma is stated for the channel-permutation group of convolutional and dense layers; a richer symmetry group has fewer invariants and would only lower the bound.

12. Ethics and Responsible Disclosure

This is offensive research carried out to strengthen defenses. Every payload in our experiments is either encrypted random bytes or a benign binary; at no point is live malware embedded, executed, or distributed. The threat we describe is against published detectors and defenses, and we will disclose our findings to the authors of Model X-Ray and NeuPerm ahead of publication, together with the mitigations of Section 10.

Reproducibility. To support defensive follow-up, we release the full code base, the quantization-aware embedding and faithful-permutation library, the detector, and the carrier model zoos, with a one-to-one mapping from every table and figure to the script that produces it. All experiments run on commodity hardware (a single GPU is optional); pretrained

models and CIFAR-10 are fetched automatically, and the diverse benign population is a 3000-model subsample of the public Model Zoos collection [29]. The code and data are released as an open GitHub repository [36] at https://github.com/robotumbel/quantghost.

13. Conclusion

QuantGhost makes a weight payload survive INT8 quantization, pruning and fine-tuning in the weights while preserving accuracy (−0.19 points on a real ResNet-56) and evading the released Model X-Ray detector, demonstrated at 3.2 MB on a real ResNet-50. Behind it sits a clean impossibility, the capacity-robustness-stealth trilemma, which for the channel-permutation group bounds any stealthy, quantization-robust permutation-invariant channel to $\sum_{\ell} k_h k_w$ bits; on a real ResNet-18 a faithful neuron permutation erases the high-capacity payload, but a 196-bit invariant channel survives and carries a key that bootstraps recovery of an out-of-band ciphertext. Deployment transforms alone, NeuPerm included, do not sanitize a shared model; doing so takes permutation together with the residual detector and the invariant-statistics inspection we propose, whose combined cost we leave to future measurement. We hope the trilemma gives defenders a concrete target.

Acknowledgment

The authors gratefully acknowledge the Media and Games Innovation Centre of Excellence (MaGIC-X), Universiti Teknologi Malaysia, for supporting this research through its technical resources and computing facilities. This work was funded by Universitas Dinamika Bangsa, Jambi, Indonesia.

References

[1] Agrawal, R., Jou, K., Obili, T., Parikh, D., Prajapati, S., Seth, Y., Sridhar, C., Zhang, N., Stamp, M., 2023. On the steganographic capacity of selected learning models. arXiv preprint arXiv:2308.15502 doi:10.48550/arXiv.2308.15502.

[2] Chaganti, R., Ravi, V., Alazab, M., Pham, T.D., 2021. Stegomalware: A systematic survey of malware hiding and detection in images, machine learning models and research challenges. arXiv preprint arXiv:2110.02504 doi:10.48550/arXiv.2110.02504.

[3] Chen, B., Wornell, G.W., 2001. Quantization index modulation: A class of provably good methods for digital watermarking and information embedding. IEEE Transactions on Information Theory 47, 1423–1443. doi:10.1109/18.923725.

[4] Cox, I.J., Kilian, J., Leighton, F.T., Shamoon, T., 1997. Secure spread spectrum watermarking for multimedia. IEEE Transactions on Image Processing 6, 1673–1687. doi:10.1109/83.650120.

[5] Dadkhah, S., Neto, E.C.P., Ferreira, R., Molokwu, R.C., Sadeghi, S., Ghorbani, A.A., 2024. CICIoMT2024: A benchmark dataset for multi-protocol security assessment in IoMT. Internet of Things 28, 101351. doi:10.1016/j.iot.2024.101351.

[6] Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L., 2009. ImageNet: A large-scale hierarchical image database, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248–255. doi:10.1109/CVPR.2009.5206848.

[7] Digregorio, G., Di Gennaro, M., Zanero, S., Longari, S., Carminati, M., 2026. On the (in)security of loading machine learning models, in: IEEE Symposium on Security and Privacy (S&P). doi:10.48550/arXiv.2509.06703.

[8] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N., 2021. An image is worth 16x16 words: Transformers for image recognition at scale, in: International Conference on Learning Representations (ICLR). doi:10.48550/arXiv.2010.11929.

[9] Dubin, R., 2023. Disarming attacks inside neural network models. IEEE Access 11, 124295–124303. doi:10.1109/ACCESS.2023.3330141.

[10] Gilkarov, D., Dubin, R., 2025. NeuPerm: Disrupting malware hidden in neural network parameters by leveraging permutation symmetry. arXiv preprint arXiv:2510.20367 doi:10.48550/arXiv.2510.20367.

[11] Gilkarov, D., Dubin, R., 2026. Model X-Ray: Detection of hidden malware in AI model weights using few shot learning. Journal of Information Security and Applications 101, 104517. doi:10.1016/j.jisa.2026.104517.

[12] Gu, T., Dolan-Gavitt, B., Garg, S., 2017. BadNets: Identifying vulnerabilities in the machine learning model supply chain. arXiv preprint arXiv:1708.06733 doi:10.48550/arXiv.1708.06733.

[13] Han, S., Mao, H., Dally, W.J., 2016. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, in: International Conference on Learning Representations (ICLR). doi:10.48550/arXiv.1510.00149.

[14] Han, S., Pool, J., Tran, J., Dally, W.J., 2015. Learning both weights and connections for efficient neural networks, in: Advances in Neural Information Processing Systems (NeurIPS). doi:10.48550/arXiv.1506.02626.

[15] He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778. doi:10.1109/CVPR.2016.90.

[16] Hitaj, D., Pagnotta, G., De Gaspari, F., Ruko, S., Hitaj, B., Mancini, L.V., Perez-Cruz, F., 2025. Do you trust your model? emerging malware threats in the deep learning ecosystem. IEEE Transactions on Dependable and Secure Computing 22. doi:10.1109/TDSC.2025.3586703.

[17] Hitaj, D., Pagnotta, G., Hitaj, B., Mancini, L.V., Perez-Cruz, F., 2022. MaleficNet: Hiding malware into deep neural networks using spread-spectrum channel coding, in: Computer Security – ESORICS, pp. 425–444. doi:10.1007/978-3-031-17143-7_21.

[18] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D., 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713. doi:10.48550/arXiv.1712.05877.

[19] Krizhevsky, A., 2009. Learning Multiple Layers of Features from Tiny Images. Technical Report. University of Toronto.

[20] Li, Y., Tondi, B., Barni, M., 2021. Spread-transform dither modulation watermarking of deep neural network. Journal of Information Security and Applications 63, 103004. doi:10.1016/j.jisa.2021.103004.

[21] Liu, K., Dolan-Gavitt, B., Garg, S., 2018a. Fine-pruning: Defending against backdooring attacks on deep neural networks, in: Research in Attacks, Intrusions, and Defenses (RAID), pp. 273–294. doi:10.1007/978-3-030-00470-5_13.

[22] Liu, T., Liu, Z., Liu, Q., Wen, W., Xu, W., Li, M., 2020. StegoNet: Turn deep neural network into a stegomalware, in: Annual Computer Security Applications Conference (ACSAC), pp. 928–938. doi:10.1145/3427228.3427268.

[23] Liu, Y., Ma, S., Aafer, Y., Lee, W.C., Zhai, J., Wang, W., Zhang, X., 2018b. Trojaning attack on neural networks, in: Network and Distributed System Security Symposium (NDSS). doi:10.14722/ndss.2018.23291.

[24] Luby, M., 2002. LT codes, in: 43rd Annual IEEE Symposium on Foundations of Computer Science (FOCS), pp. 271–280. doi:10.1109/SFCS.2002.1181950.

[25] Moulin, P., O'Sullivan, J.A., 2003. Information-theoretic analysis of information hiding. IEEE Transactions on Information Theory 49,

563–593. doi:10.1109/TIT.2003.809000.

[26] Nagel, M., Fournarakis, M., Amjad, R.A., Bondarenko, Y., van Baalen, M., Blankevoort, T., 2021. A white paper on neural network quantization. arXiv preprint arXiv:2106.08295 doi:10.48550/arXiv.2106.08295.

[27] Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A., 2023. CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors 23, 5941. doi:10.3390/s23135941.

[28] Pagnotta, G., Hitaj, D., Hitaj, B., Perez-Cruz, F., Mancini, L.V., 2022. TATTOOED: A robust deep neural network watermarking scheme based on spread-spectrum channel coding. arXiv preprint arXiv:2202.06091 doi:10.48550/arXiv.2202.06091.

[29] Schürholt, K., Taskiran, D., Knyazev, B., Giró-i Nieto, X., Borth, D., 2022. Model zoos: A dataset of diverse populations of neural network models, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track. doi:10.48550/arXiv.2209.14764.

[30] Snell, J., Swersky, K., Zemel, R., 2017. Prototypical networks for few-shot learning, in: Advances in Neural Information Processing Systems (NeurIPS). doi:10.48550/arXiv.1703.05175.

[31] Tondi, B., Costanzo, A., Barni, M., 2022. Robust and large-payload DNN watermarking via fixed, distribution-optimized, weights. arXiv preprint arXiv:2208.10973 doi:10.48550/arXiv.2208.10973.

[32] Torpmann-Hagen, B., Riegler, M.A., Halvorsen, P., Johansen, D., 2025. Defending against stegomalware in deep neural networks with permutation symmetry. arXiv:2509.20399. doi:10.48550/arXiv.2509.20399.

[33] Vinyals, O., Blundell, C., Lillicrap, T., Kavukcuoglu, K., Wierstra, D., 2016. Matching networks for one shot learning, in: Advances in Neural Information Processing Systems (NeurIPS). doi:10.48550/arXiv.1606.04080.

[34] Wang, Z., Liu, C., Cui, X., 2021. EvilModel: Hiding malware inside of neural network models, in: IEEE Symposium on Computers and Communications (ISCC), pp. 1–7. doi:10.1109/ISCC53001.2021.9631425.

[35] Wang, Z., Liu, C., Cui, X., Yin, J., Wang, X., 2022. EvilModel 2.0: Bringing neural network models into malware attacks. Computers & Security 120, 102807. doi:10.1016/j.cose.2022.102807.

[36] Winanto, E.A., Idris, M.Y., Kurniabudi, 2026. QuantGhost: Deployment-robust, stealthy weight steganography—code and data. GitHub repository. https://github.com/robotumbel/quantghost.

[37] Zhang, L., Li, D., Jurečková, O., Stamp, M., 2023. Steganographic capacity of deep learning models. arXiv preprint arXiv:2306.17189 doi:10.48550/arXiv.2306.17189.

[38] Zhu, J., Jang-Jaccard, J., Singh, A., Watters, P.A., Camtepe, S., 2023. Task-aware meta learning-based siamese neural network for classifying control flow obfuscated malware. Future Internet 15, 214. doi:10.3390/fi15060214.

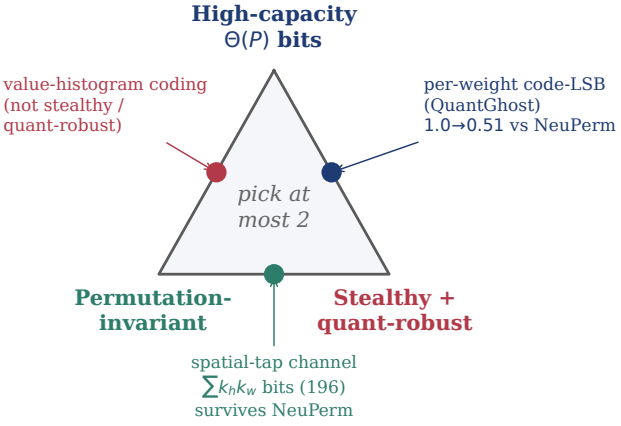


Figure 2: The trilemma. Each embedding occupies one edge, securing the two adjacent properties and giving up the opposite vertex.

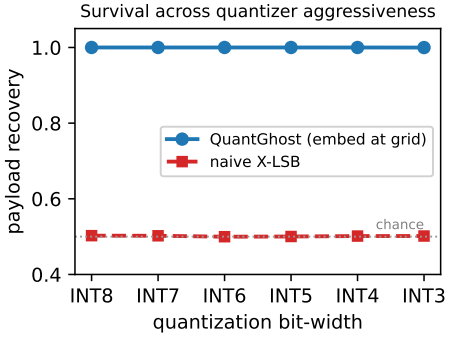


Figure 3: Payload recovery versus quantizer bit-width (chance 0.5). QuantGhost stays at 1.000 from INT8 to INT3 when embedded on the deployment grid; the naive mantissa-LSB attack is erased at every width.

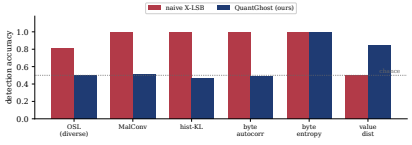


Figure 4: Detection accuracy (chance 0.5) of the naive attack and ours across six detectors. Ours evades four; only byte-entropy and value-distribution catch it.

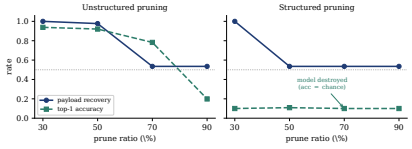


Figure 5: Pruning on a real ResNet-56: payload recovery and top-1 accuracy versus ratio. The payload only dies where accuracy also dies; structured pruning destroys the model at every ratio.

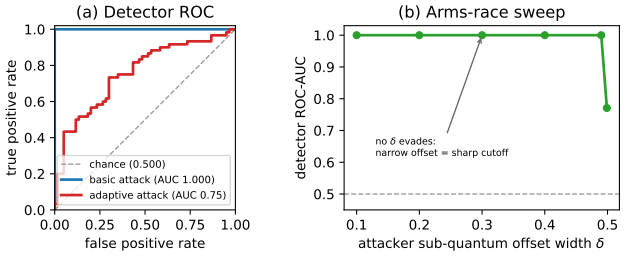


Figure 6: Proposed sub-quantum residual detector. (a) ROC curves (out-of-fold scores): AUC 1.000 against the basic attack, 0.75 against the adaptive attack, both far above the chance diagonal. (b) Arms-race sweep on the carrier zoo: detector AUC versus the attacker's sub-quantum offset width δ; no interior δ evades. On a diverse zoo (Table 13) the adaptive attacker instead reaches chance.